



Operadores e Condicionais em Python

Bem-vindo à aula de **Algoritmos**! Nesta aula, vamos explorar os operadores fundamentais da linguagem Python e aprender a controlar o fluxo de execução dos nossos programas com estruturas condicionais. Utilizaremos o **VSCode** como ambiente de desenvolvimento.

ALGORITMOS II

PYTHON + VSCODE

Objetivo da Aula

Ao final desta aula, você será capaz de utilizar estruturas condicionais para tomada de decisão em seus programas Python.

1

Compreender Operadores

Conhecer os operadores aritméticos, relacionais e lógicos disponíveis em Python e como aplicá-los em expressões.

2

Estrutura if / elif / else

Utilizar as estruturas condicionais para controlar o fluxo de execução do programa com base em condições avaliadas em tempo de execução.

3

Aplicar em Exemplos Reais

Praticar a lógica condicional com exemplos concretos e resolver exercícios da lista prática para fixação do conteúdo.

Operadores Aritméticos

Operadores aritméticos são usados para realizar cálculos matemáticos. O Python oferece um conjunto completo deles, incluindo divisão inteira e potenciação.

+ Adição

Soma dois valores.

$3 + 2 \rightarrow 5$

- Subtração

Subtrai o segundo do primeiro.

$5 - 2 \rightarrow 3$

* Multiplicação

Multiplica dois valores.

$4 * 3 \rightarrow 12$

/ Divisão

Divisão real (float).

$7 / 2 \rightarrow 3.5$

// Divisão Inteira

Retorna apenas a parte inteira.

$7 // 2 \rightarrow 3$

% Módulo

Retorna o resto da divisão.

$7 \% 2 \rightarrow 1$

** Potência

Eleva à potência indicada.

$2 ** 3 \rightarrow 8$



Dica: O operador `%` (módulo) é muito utilizado para verificar se um número é par ou ímpar: `n % 2 == 0` indica que `n` é par.

Operadores Relacionais

Operadores relacionais comparam dois valores e retornam um resultado booleano: True ou False. Eles são a base das condições usadas em estruturas if. Compreender cada operador é essencial para escrever condições corretas e precisas.

== Igualdade

Verifica se dois valores são **iguais**.

5 == 5 → True

5 == 3 → False

!= Diferença

Verifica se dois valores são **diferentes**.

5 != 3 → True

5 != 5 → False

> Maior que

Verdadeiro se o primeiro é **maior** que o segundo.

7 > 3 → True

< Menor que

Verdadeiro se o primeiro é **menor** que o segundo.

2 < 5 → True

>= Maior ou igual

Verdadeiro se o primeiro é **maior ou igual** ao segundo.

5 >= 5 → True

<= Menor ou igual

Verdadeiro se o primeiro é **menor ou igual** ao segundo.

3 <= 4 → True

📌 ⚠️ **Atenção:** Não confunda = (atribuição de valor a uma variável) com == (comparação de igualdade). Este é um dos erros mais comuns entre iniciantes!

Operadores Lógicos

Os operadores lógicos permitem **combinar múltiplas condições** em uma única expressão. Em Python, temos três operadores lógicos principais, que operam sobre valores booleanos (True / False).

and — E lógico

Retorna True somente quando **todas** as condições são verdadeiras. Se qualquer uma for falsa, o resultado é False.

`5 > 3 and 2 < 4 → True`

`5 > 3 and 2 > 4 → False`

or — OU lógico

Retorna True quando **pelo menos uma** das condições é verdadeira. Só retorna False quando todas são falsas.

`5 > 3 or 2 > 4 → True`

`1 > 3 or 2 > 4 → False`

not — NÃO lógico

Inverte o valor booleano de uma condição. Se era True, vira False, e vice-versa.

`not True → False`

`not (5 > 3) → False`

Estrutura if

A estrutura `if` é a forma mais simples de tomada de decisão em Python. Ela avalia uma condição: se a condição for **verdadeira** (`True`), o bloco de código indentado é executado. Se for **falsa** (`False`), o bloco é simplesmente ignorado.

Sintaxe

```
if condição:  
    código a executar
```

A condição pode ser qualquer expressão que resulte em `True` ou `False`, incluindo comparações com operadores relacionais e lógicos.

Como funciona

- 1 Avaliação da condição**
Python avalia a expressão após o `if`. O resultado deve ser booleano.
- 2 Execução condicional**
Se `True`, o bloco indentado é executado. Se `False`, é pulado.
- 3 Continuação do programa**
O programa continua normalmente após o bloco `if`.

📌 **Regra fundamental:** Em Python, a indentação (4 espaços ou 1 TAB) delimita o bloco de código que pertence ao `if`. Erros de indentação causam falhas na execução do programa!

Estrutura Completa: if / elif / else

Na prática, muitos problemas exigem mais de duas possibilidades. A estrutura completa combina `if`, `elif` (abreviação de *else if*) e `else` para cobrir múltiplos cenários de forma organizada e legível.

1

`if` condição1:

Primeira condição avaliada. Se for `True`, executa este bloco e **pula** todo o restante da estrutura.

2

`elif` condição2:

Avaliado **somente** se o `if` anterior foi `False`. Podem existir múltiplos blocos `elif` em sequência para cobrir diferentes casos.

3

`else`:

Executado quando **nenhuma** das condições anteriores foi verdadeira. É opcional, mas recomendado para garantir um comportamento padrão.

`if` condição1:

bloco 1

`elif` condição2:

bloco 2

`else`:

bloco padrão



Exemplo 1: Verificação de Idade

Vamos aplicar o `if / else` em um caso prático: verificar se uma pessoa é maior ou menor de idade com base no valor digitado pelo usuário.

Código Python

```
idade = int(input("Idade: "))

if idade >= 18:
    print("Maior de idade")
else:
    print("Menor de idade")
```

Entendendo o código

→ `input()` lê a entrada do usuário

A função `input()` sempre retorna uma `string`. Por isso é necessário converter com `int()`.

→ **Condição** `idade >= 18`

Usa o operador relacional "maior ou igual". Se a idade for 18 ou mais, a condição é verdadeira.

→ **Bloco else**

Cobre todos os demais casos — ou seja, idades menores que 18 anos.

Saídas possíveis

Entrada	Saída
20	Maior de idade
18	Maior de idade
15	Menor de idade

Exemplo 2: Sistema de Notas

Agora utilizamos a estrutura completa if / elif / else para classificar o desempenho de um aluno com base na nota informada. Este é um exemplo clássico com três saídas possíveis.

Código Python

```
nota = float(input("Nota: "))

if nota >= 7:
    print("Aprovado")
elif nota >= 5:
    print("Recuperação")
else:
    print("Reprovado")
```

Fluxo de decisão

01

Nota >= 7.0?

Se True → imprime "**Aprovado**" e encerra a estrutura.

02

Nota >= 5.0?

Avaliado apenas se o primeiro if foi falso. Se True → imprime "**Recuperação**".

03

Nenhuma condição verdadeira

O bloco else é executado → imprime "**Reprovado**".

Tabela de resultados

Nota	Resultado
8.5	Aprovado ✓
6.0	Recuperação ⚠
3.0	Reprovado ✗



💡 **Por que usar float()?** Notas frequentemente têm casas decimais (ex: 6.5, 7.8). Usar int() truncaria esses valores e causaria resultados incorretos.

Boas Práticas de Codificação

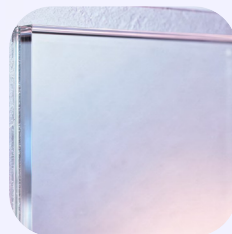
Escrever código que funciona é apenas o primeiro passo. Escrever código **legível, organizado e correto** é o que diferencia um bom programador. Duas práticas são especialmente críticas ao trabalhar com condicionais em Python.



Indentação Correta

Em Python, a indentação **não é opcional** — ela é parte da sintaxe da linguagem. Use **4 espaços** (ou 1 TAB configurado para 4 espaços no VSCode) para cada nível de aninhamento.

- Nunca misture espaços e tabs no mesmo arquivo
- O VSCode pode ser configurado para converter TAB em 4 espaços automaticamente
- Um erro de indentação gera `IndentationError` na execução



Clareza nas Condições

Condições devem ser **fáceis de ler e entender**. Evite expressões excessivamente complexas ou negações desnecessárias. Prefira legibilidade à brevidade.

- Use nomes de variáveis descritivos (`idade`, `nota`, não `x`, `n`)
- Evite condições como `not not valor` — simplifique
- Use parênteses para deixar a precedência explícita em condições compostas
- Prefira `if saldo >= 0`: a `if not saldo < 0`:

Executando o Programa

Com o código pronto e salvo no VSCode, é hora de executar o arquivo Python. Você pode rodar o programa de duas maneiras diferentes.

Via Terminal (Recomendado)

Abra o terminal integrado no VSCode com `Ctrl + `` e execute:

```
python aula_condicional.py
```

Certifique-se de estar na mesma pasta do arquivo. Você pode verificar com `ls` (Linux/Mac) ou `dir` (Windows).

Passo a passo no VSCode

01

Salve o arquivo

Use `Ctrl + S`. Certifique-se que o nome termina em `.py`.

02

Abra o terminal

Menu **Terminal** → **New Terminal** ou atalho `Ctrl + ``.

03

Execute o comando

Digite `python aula_condicional.py` e pressione Enter.

04

Interaja com o programa

Caso haja `input()`, o terminal aguardará a sua digitação.



No Windows, pode ser necessário usar `python3` ou `py` em vez de `python`.

Exercícios Práticos

A melhor forma de fixar o conteúdo é **praticar**! Resolva a lista de exercícios abaixo utilizando os operadores e condicionais aprendidos nesta aula. Teste cada programa no VSCode e experimente diferentes entradas para validar seu código.

1

Par ou Ímpar

Leia um número inteiro e informe se ele é **par** ou **ímpar**. Use o operador `%`.

2

Positivo, Negativo ou Zero

Leia um número e classifique-o em **positivo**, **negativo** ou **zero** usando `if / elif / else`.

3

Maior de Três

Leia três números e informe qual é o **maior** entre eles. Use operadores relacionais e lógicos.

4

Calculadora Simples

Leia dois números e uma operação (`+`, `-`, `*`, `/`) e exiba o resultado usando condicionais.



Dica de estudo: Antes de escrever o código, desenhe o fluxo lógico no papel. Identifique quais são as condições e os casos possíveis. Isso reduz erros e torna o desenvolvimento mais rápido!